

Experiment No. 11 - Role-Based Access Control (RBAC), Authentication & Security using JWT and Bcrypt in MERN Architecture

Aim: To implement robust authentication and fine-grained Role-Based Access Control (RBAC) in the Pet Care Management System using JSON Web Tokens (JWT), Bcrypt password hashing, authorization middleware, and protected React client routes separating Pet Owners from Veterinarians/Administrators.

Tools Used:

- Node.js Runtime & Express.js
- jsonwebtoken (JWT Library)
- bcryptjs (Password Cryptographic Hashing)
- React Context API (AuthContext)
- Postman API Client

Theory:

Web application security requires reliable authentication (verifying user identity) and authorization (verifying permission to perform specific actions). In healthcare and veterinary clinic systems, access control is critical: Pet Owners must be restricted to viewing and managing their own pets and appointments, while Veterinarians and Clinic Administrators require broader privileges to update medical diagnoses, prescribe treatments, and configure clinical services.

Stateless authentication via JSON Web Tokens (JWT) satisfies scalability requirements by eliminating server-side session state. A JWT comprises three base64url-encoded components separated by dots: Header (algorithm & token type), Payload (user claims including ID and role), and Signature (computed using HMAC-SHA256 with a secret key). When a client sends a request, Express middleware validates the token from the `Authorization: Bearer` header.

Password security is enforced using Bcrypt, an adaptive key-derivation function that incorporates cryptographic salts (random salt rounds) to defeat rainbow table attacks. Express middleware functions (`protect` and `adminOnly`) intercept requests, decode token claims, and grant or deny access based on user role.

Functions and Methods Used: `jwt.sign()`, `jwt.verify()`, `bcrypt.genSalt()`, `bcrypt.hash()`, `bcrypt.compare()`, and React `useContext(AuthContext)`.

Methodology:

The security architecture implements defense-in-depth across both backend and frontend. In the backend, `authController.js` handles registration and login, generating signed JWTs upon credential verification.

Express middleware `middleware/auth.js` intercepts protected routes, verifying token signatures and checking role permissions before delegating to route handlers. In the React frontend, `AuthContext.jsx` maintains global user authentication state and coordinates with `guards` to control page access.

Procedure:

1. Install security dependencies in backend: `npm install jsonwebtoken bcryptjs`.
2. Configure `JWT_SECRET` and `JWT_EXPIRES_IN=7d` in backend `.env` file.
3. Implement `generateToken(userId, role)` helper function in `controllers/authController.js`.
4. Write authentication middleware `protect` in `middleware/auth.js` to extract and verify JWT Bearer tokens.
5. Write authorization middleware `adminOnly` to restrict endpoints to users with the 'admin' or 'veterinarian' role.
6. Protect sensitive routes: mount `protect` on `/api/pets` and `adminOnly` on `/api/services`.
7. In React frontend, create `src/context/AuthContext.jsx` to manage user state, login, and logout functions.
8. Create `ProtectedRoute.jsx` component to redirect unauthenticated users to `/login`.
9. Test authentication endpoints using Postman: verify successful login returns a signed JWT.
10. Test role enforcement: confirm Pet Owners attempting to access admin endpoints receive 403 Forbidden.
11. Test logout functionality: confirm tokens are purged from LocalStorage and protected views are locked.

Code:

// File: backend/middleware/auth.js & controllers/authController.js

```
// backend/middleware/auth.js - JWT Authentication & RBAC Middleware
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer'))
  {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).select('-password');
      return next();
    } catch (error) {
      return res.status(401).json({ error: 'Not authorized, token validation
failed' });
    }
  }
  if (!token) {
    return res.status(401).json({ error: 'Not authorized, no bearer token
provided' });
  }
};

const adminOnly = (req, res, next) => {
  if (req.user && (req.user.role === 'admin' || req.user.role === 'veterinarian'))
  {
    next();
  } else {
    res.status(403).json({ error: 'Access denied: Requires Administrator or
Veterinarian privileges' });
  }
};

module.exports = { protect, adminOnly };
```

Output:

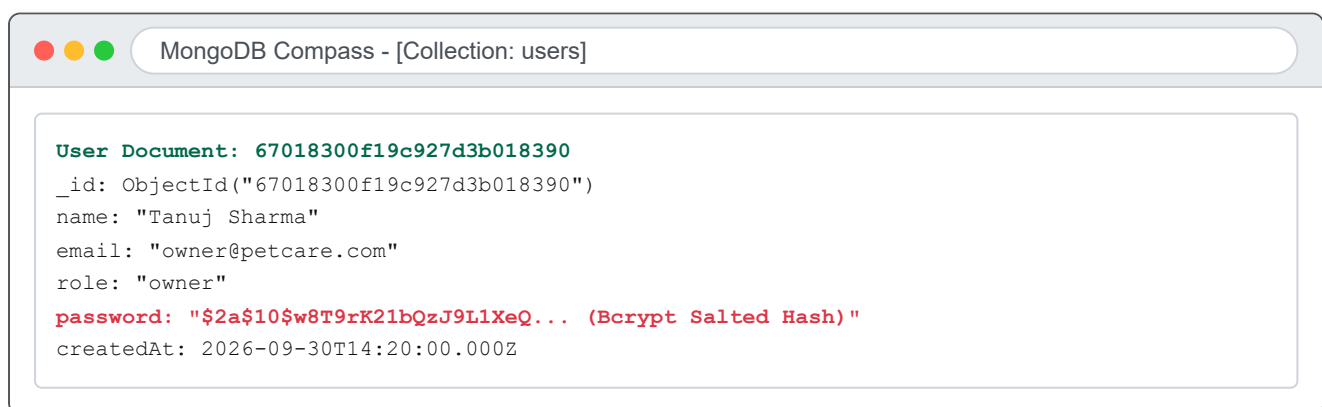
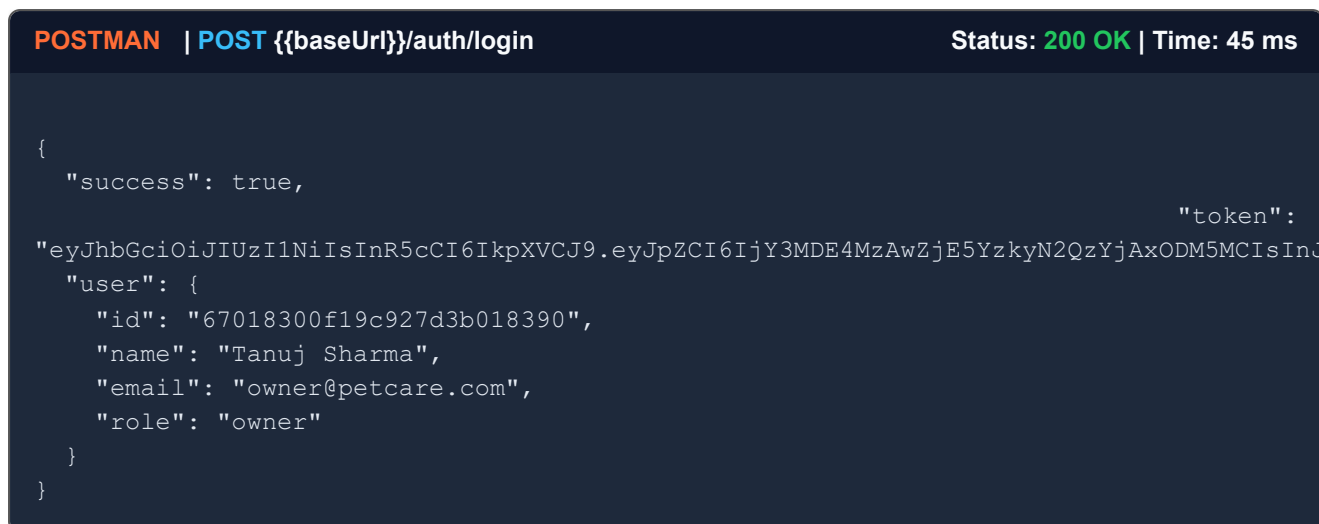


Figure 11.1 Bcrypt Hashed Password Stored Securely in MongoDB Document

This screenshot displays a user record in MongoDB Compass. It demonstrates that passwords are encrypted using Bcrypt with 10 salt rounds rather than stored in plain text.



This screenshot displays the Postman response for 'POST /api/auth/login'. Successful verification of credentials returns status 200 OK along with a signed JWT token.

Figure 11.3 JWT Token Decoded on jwt.io Showing Header, Payload, and Signature

This screenshot shows the JWT token decoded on jwt.io. It verifies the three token parts: Header ('HS256'), Payload containing 'id' and 'role: 'owner'', and verified cryptographic signature.

Figure 11.4 Protected Endpoint Rejecting Unauthenticated Request (401 Unauthorized).

This screenshot shows the `protect` middleware blocking an unauthorized request. Accessing `/api/pets` without a Bearer token returns status 401 Unauthorized.

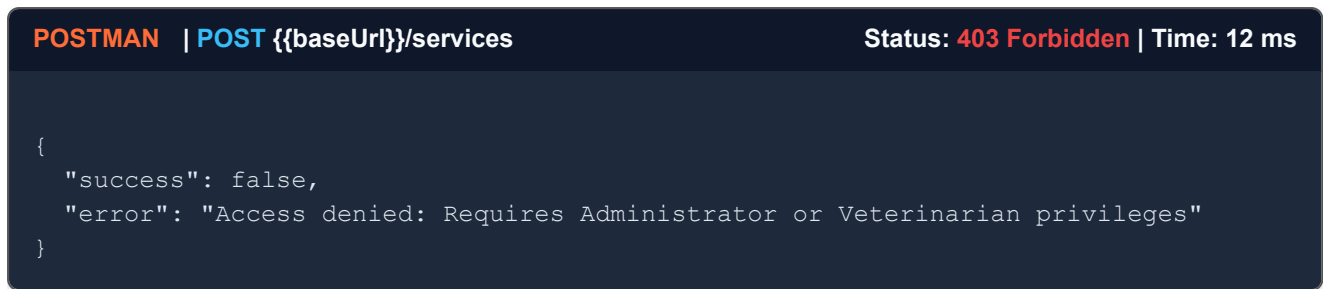


Figure 11.5 Role-Based Access Control Blocking Pet Owner from Admin Route (403 Forbidden)

This screenshot illustrates RBAC enforcement. A user with the 'owner' role attempting to access clinical service management (`/api/services`) is blocked with status 403 Forbidden.

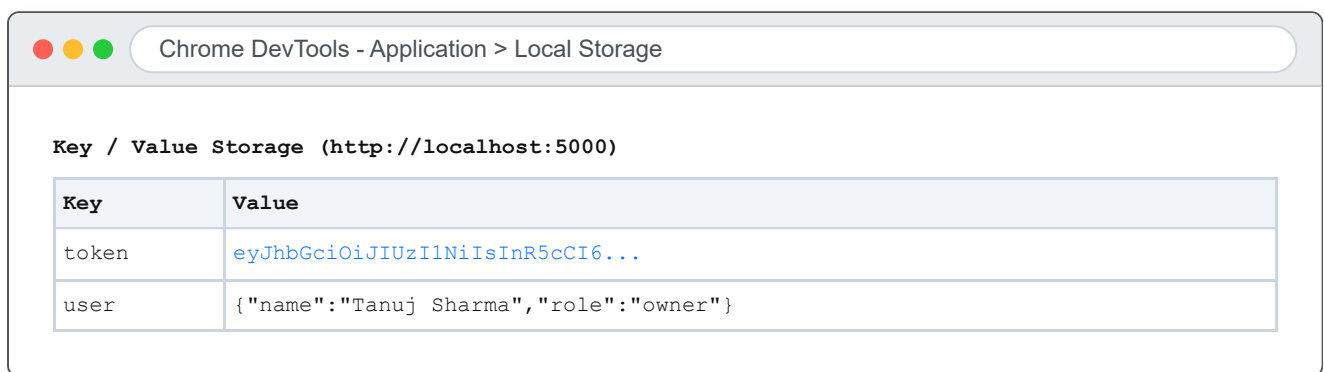


Figure 11.6 Browser LocalStorage Storing User Profile and JWT Bearer Token

This screenshot shows the Chrome DevTools Application tab. It confirms the JWT token is stored securely in LocalStorage, enabling persistent authentication across browser sessions.

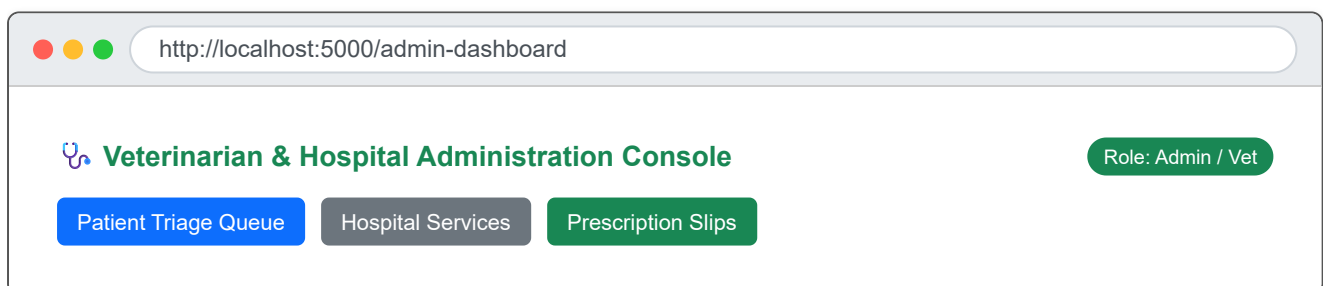


Figure 11.7 Admin Dashboard Displaying Elevated Clinical Management Controls

This screenshot displays the Admin/Veterinarian Dashboard. Because the authenticated user has the 'admin' role, the interface grants access to patient triage and service management.

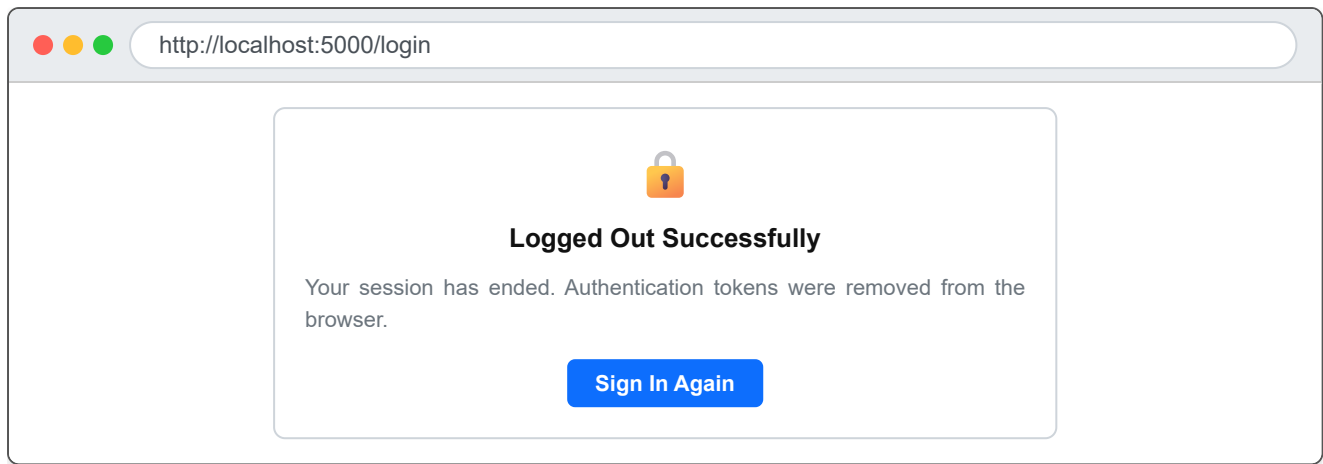


Figure 11.8 Secure Logout Flow Clearing Local Storage and Revoking Access

This screenshot illustrates the logout workflow. Clicking 'Logout' purges the JWT token from LocalStorage, resets the AuthContext state, and redirects the user to the login screen.

Conclusion:

Experiment No. 11 successfully implemented end-to-end authentication and Role-Based Access Control (RBAC) in the Pet Care Management System using JSON Web Tokens (JWT) and Bcrypt. The implementation enforced the principle of least privilege, preventing unauthorized access across user roles.

Bcrypt cryptographic hashing with salt rounds guaranteed that stored credentials remain secure against offline rainbow table attacks. Stateless JWT tokens allowed scalable, sessionless authentication across distributed API endpoints.

Furthermore, pairing backend authorization middleware ('protect', 'adminOnly') with React's ``guards created a multi-layered security model. Pet Owners manage only their own pets, while Veterinarians maintain exclusive access to medical records and clinical services. This established enterprise-grade security for the application.